

ПАРАЛЛЕЛИЗМ НА УРОВНЕ ДАННЫХ

Архитектура процессоров и технология CUDA

Перцев Дмитрий – 18.09.2026

Содержание лекции



Часть I. Введение в векторизацию

1. SIMD-инструкции и параллелизм на уровне данных
2. Что такое векторизация?
3. Развертка цикла
4. Условия автоматической векторизации

Часть II. Векторные расширения ISA

1. Типы векторных инструкций
2. Векторные расширения x86
3. AVX и AVX-512

Часть III. Практические примеры

1. Примеры кода и asm-листинги



Часть IV. Векторные архитектуры

1. Примеры векторных процессоров
2. NEC SX-Aurora TSUBASA

ВВЕДЕНИЕ В ВЕКТОРИЗАЦИЮ

SIMD-инструкции



- SIMD - частный случай автоматического параллелизма на уровне данных
  - ориентированы на массивы данных
  - медиа-ориентированные процессоры для обработки изображений и звука
- SIMD более энергоэффективен, чем MIMD
  - на одну операцию с данными требуется только поток одной инструкции
  - это делает SIMD привлекательной для персональных и мобильных устройств
- SIMD **позволяет** программам **мыслить последовательно**

Параллелизм на уровне данных



- векторные архитектуры
- SIMD расширения
- графические процессоры (ГПУ)

Что такое векторизация?



- это частный случай автоматического распараллеливания кода, использующий вместо скалярных векторные инструкции
- может выполняться компилятором или человеком
- возможности векторизации компилятором ограничены

Сложности при разработке векторного процессора



- вектор не кратной ширины
- операции над векторным кодом
- оптимизация системы памяти для поддержки векторных процессоров
- многомерные матрицы
- разреженные матрицы
- программирование векторного сопроцессора



РАЗВЕРТКА ЦИКЛА И ВЕКТОРИЗАЦИЯ

Развертка циклов



- подход к повышению производительности
  - уменьшается количество исполняемых инструкций, связанных с вычислением и проверкой условия
- пример:

Исходный код	Развернутый код

`for (int i = 0; i < N; i++) {`
`  function();`
`}`
	`for (int i = 0; i < N/2; i += 2) {`
	`  function();`
	`  function();`
	`}`

Пример развертки цикла

```
loop:
  ld r4, [r1]
  ld r5, [r2]
  add r4, r4, r5
  st [r3], r4
  add r1, r1, 4
  add r2, r2, 4
  add r3, r3, 4
  bne r1, r10, loop
```

```
loop:
  ld r4, [r1]
  ld r5, [r2]
  add r4, r4, r5
  st [r3], r4
  ld r4, [r1+4]
  ld r5, [r2+4]
  add r4, r4, r5
  st [r3+4], r4
  add r1, r1, 8
  add r2, r2, 8
  add r3, r3, 8
  bne r1, r10, loop
```

Необходимые условия для развертки цикла



- цикл for
  - необходимо знать функцию вычисления индекса
- количество итераций цикла не должно зависеть от данных
- неизвестно во время компиляции количество итераций цикла
  - генерируется код, состоящий из развернутой и обычной частей
    - например, степень развертки 4
    - количество итераций цикла 22
    - 16 итераций исполняются двумя итерациями развернутого цикла
    - 6 итераций исполняются на обычном цикле

Недостатки развертки циклов



- увеличение количества инструкций приводит к увеличению вероятности кэш промахов
- может не хватить регистров

Как выполняется векторизация



- векторизация основана на развертке цикла
- выполняется только для операций, которые находятся внутри цикла
- скалярные элементы объединяются в вектора и скалярные операции заменяются векторными

Условия автоматической векторизации



- операция находится в цикле
- итерации цикла не зависимы
- число итераций в цикле не должно зависеть от данных
- типы данных и операции, которые должны быть векторизованы, должны аппаратно поддерживаться
- внутри цикла не должно быть вызовов процедур
- шаг между элементами в памяти равен 1

Векторные инструкции в различных архитектурах



- Архитектура x86 (x86-64)
  - MMX, 3DNow!
  - SSE-SSE4.x, SSSE3
  - AVX, AVX2, AVX-512
- Архитектура PowerPC
  - AltiVec, SPE
- Архитектура SPARC
  - VIS
- Архитектура ARM
  - NEON



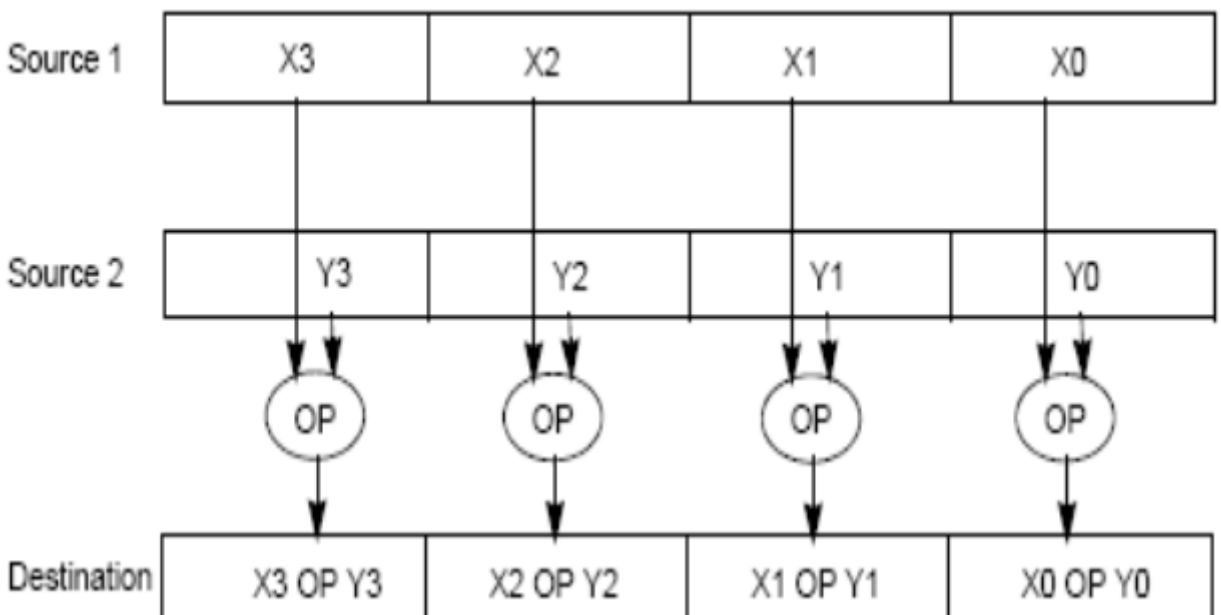
ВЕКТОРНЫЕ РАСШИРЕНИЯ ISA

Типы векторных инструкций

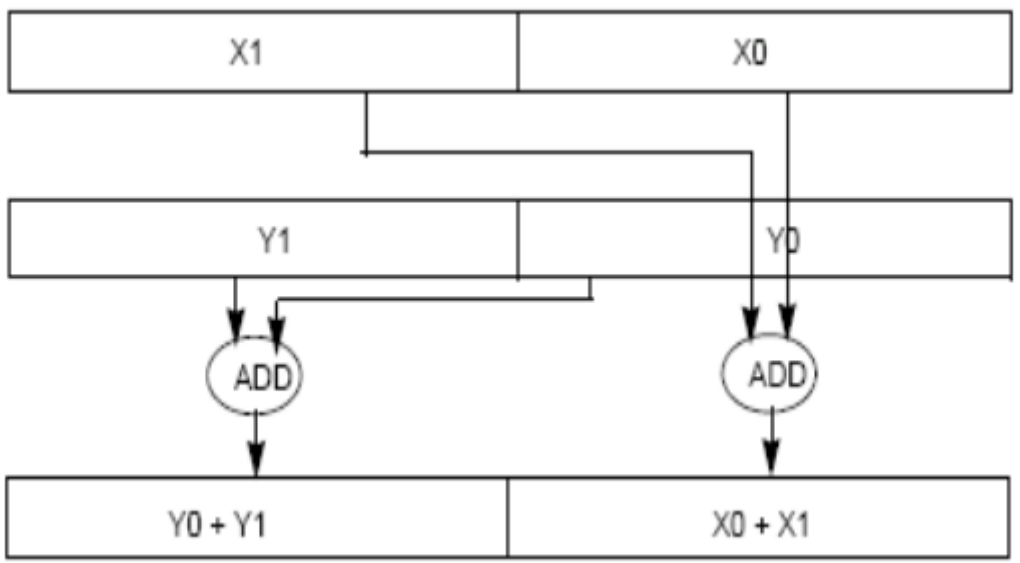


- вертикальные
- горизонтальные
- комбинированные

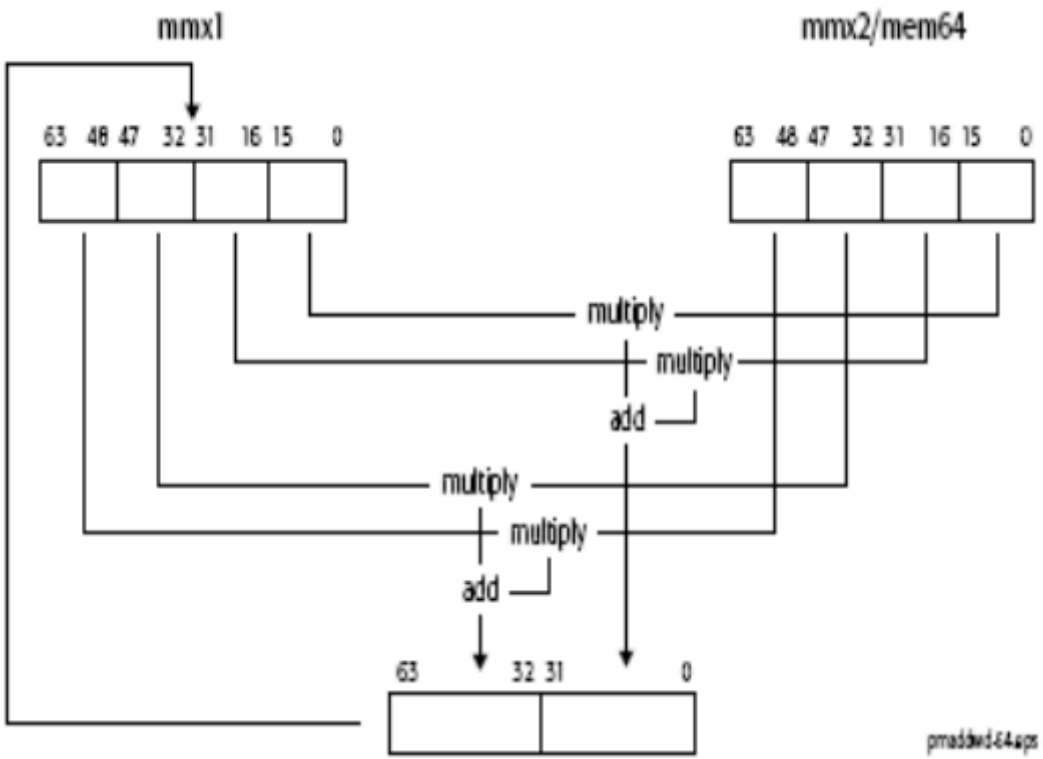
Вертикальные векторные инструкции



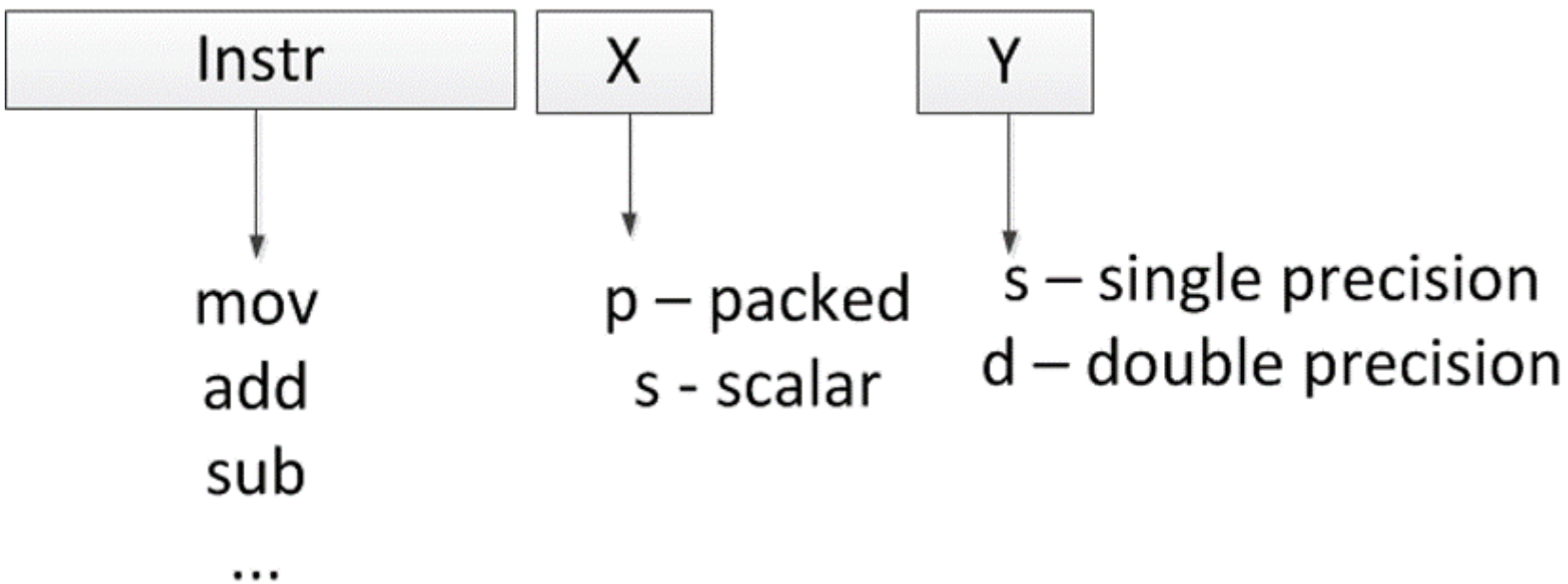
Горизонтальные векторные инструкции



Комбинированные векторные инструкции



Формат инструкций



Проблемы инструкций копирования



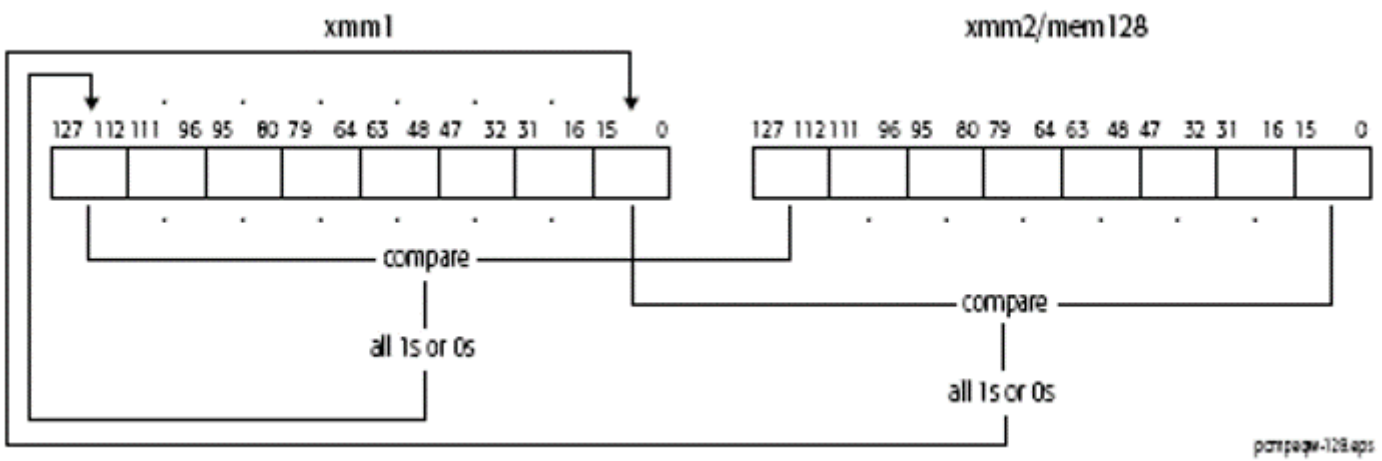
- `movUps` - копирование не выравненных данных
  - не до 2 обращений в память вместо одного
- `movSs` - скалярное (S) копирование

Целочисленные векторные инструкции x86

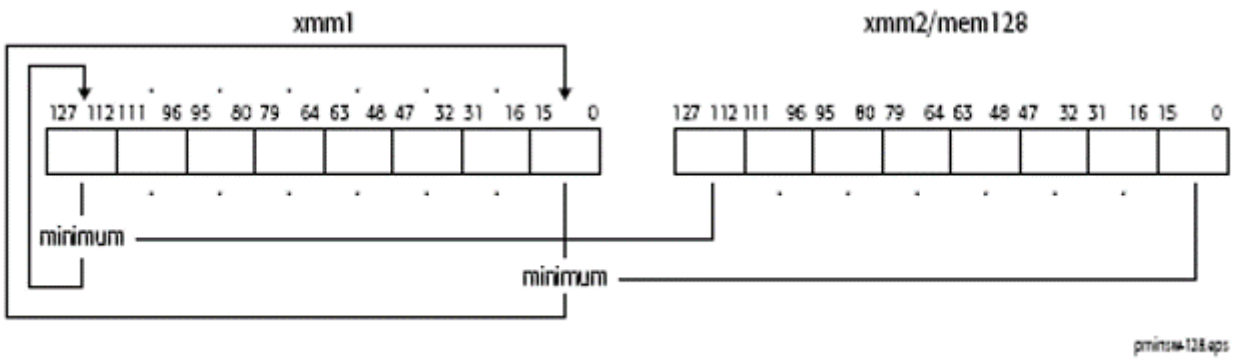


- целочисленные инструкции входят в наборы MMX, SSE2, SSE3, SSSE3, SSE4.1, AVX2
- в качестве аргументов инструкции могут использовать
  - 64-битные регистры MMX
  - 128-ми битные регистры XMM
  - 256-битные регистры YMM
- поддерживается два типа арифметики:
  - с насыщением, без насыщения (циклическая)

Целочисленные векторные инструкции x86 (инструкция PMADDWD)



Целочисленные векторные инструкции x86 (инструкция PMAXSW)

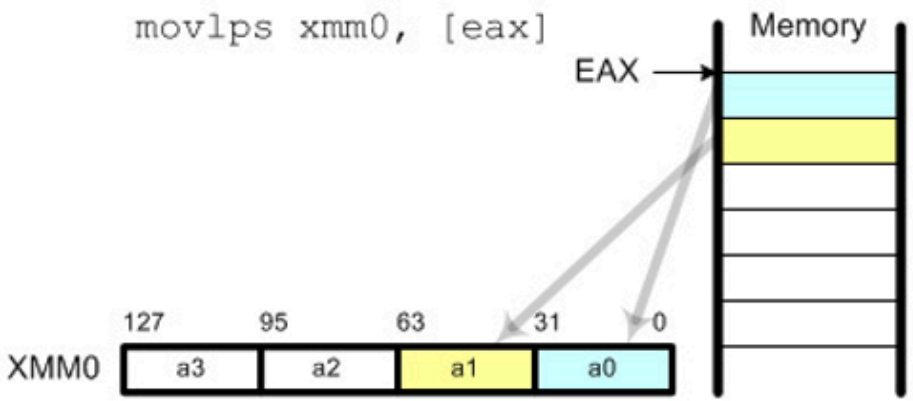


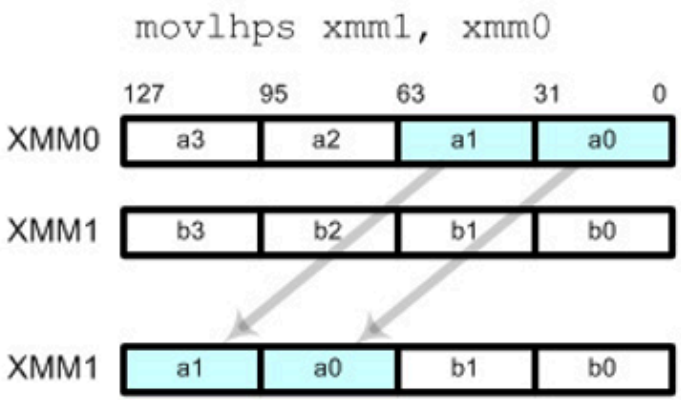
Инструкция BLEND

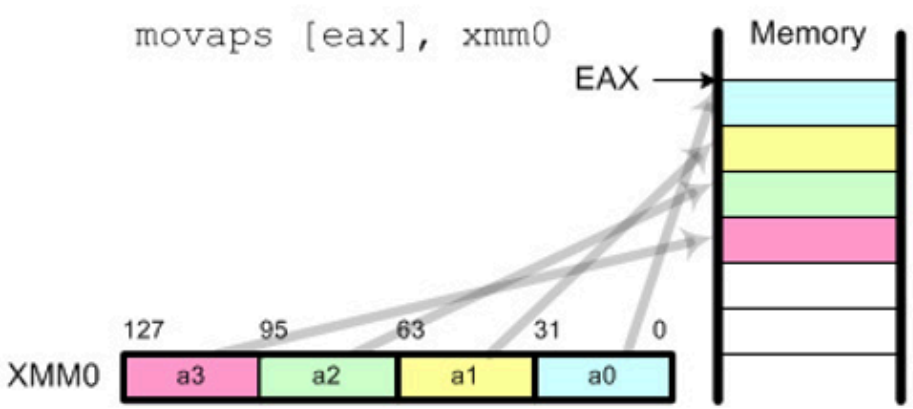


- выполнение операций слияния и разделения при условной обработке векторов

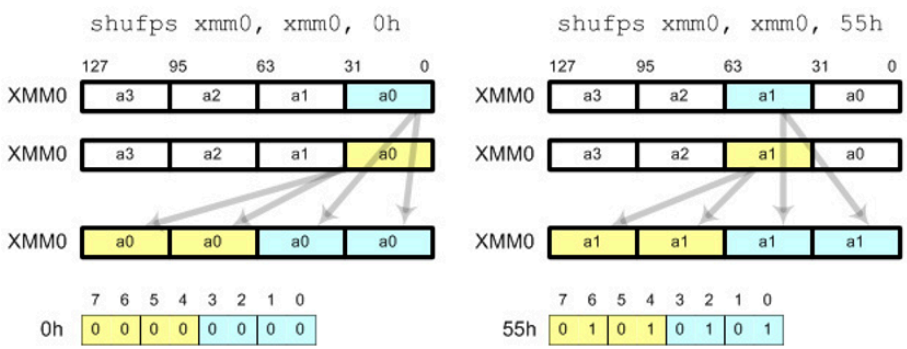
Инструкции копирования





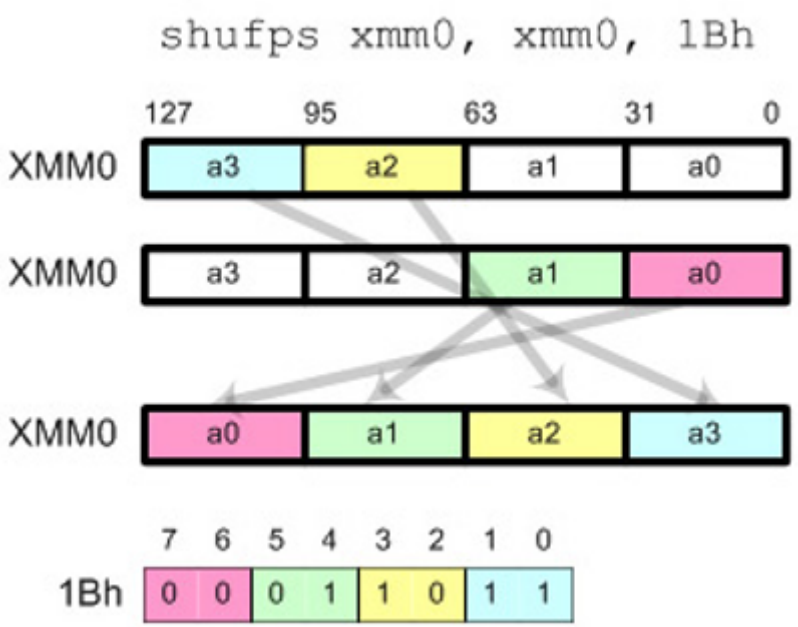


Инструкция shuffle

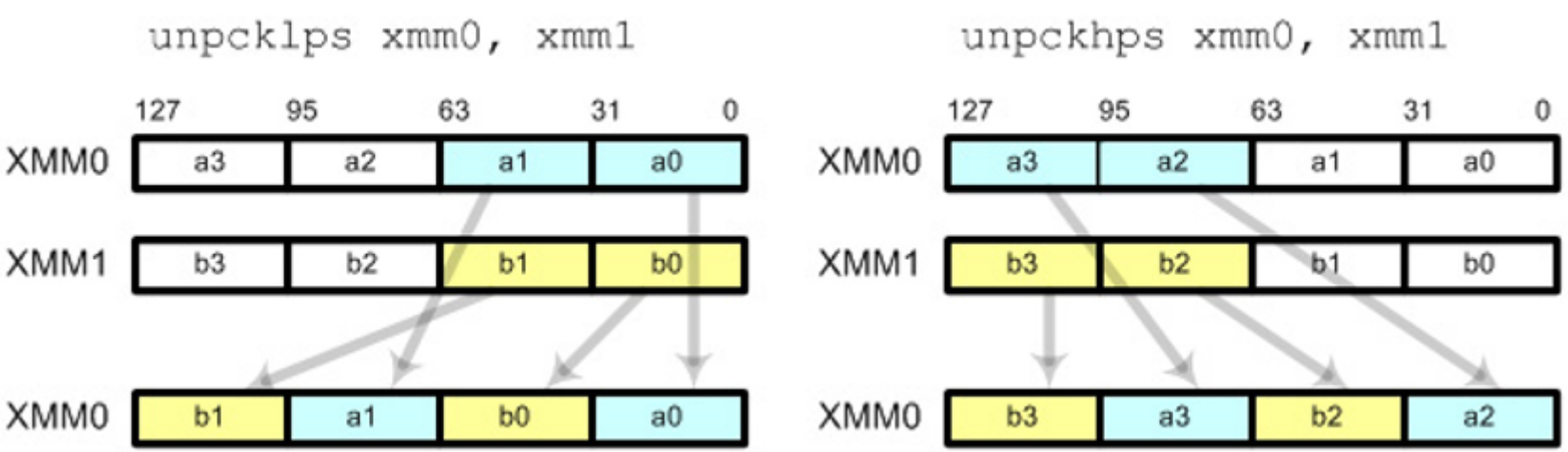


- 00h: копирует младший элемент
- 55h: копирует второй элемент
- AAh: копирует третий элемент
- FFh: копирует четвертый элемент

Своп данных


shufps xmm0, xmm0, 1Bh
127
95
63
31
0
XMM0
a3
a2
a1
a0
XMM0
a3
a2
a1
a0
XMM0
a0
a1
a2
a3
7 6 5 4 3 2 1 0
1Bh
0 0 0 1 1 0 1 1

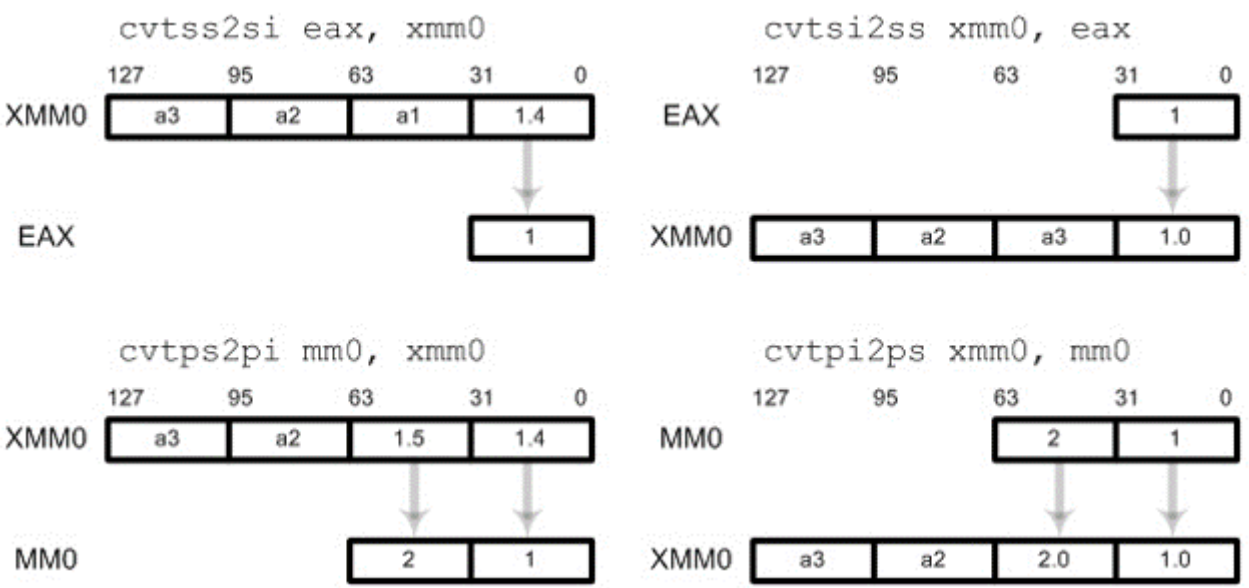
Unpack


unpcklps xmm0, xmm1
127
95
63
31
0
XMM0
a3
a2
a1
a0
XMM1
b3
b2
b1
b0
XMM0
b1
a1
b0
a0
unpckhps xmm0, xmm1
127
95
63
31
0
XMM0
a3
a2
a1
a0
XMM1
b3
b2
b1
b0
XMM0
b3
a3
b2
a2

Конверсия данных



Conversion	Scalar Operation	Packed Operation
Float to integer with rounding	`cvttss2si`	`cvttps2pi`
Float to integer with truncation	`cvtss2si`	`cvtps2pi`
Integer to float	`cvtsi2ss`	`cvtpi2ps`


cvtss2si eax, xmm0
127
95
63
31
0
XMM0
a3
a2
a1
1.4
EAX
1
cvtsi2ss xmm0, eax
127
95
63
31
0
EAX
1
XMM0
a3
a2
a3
1.0
cvtps2pi mm0, xmm0
127
95
63
31
0
XMM0
a3
a2
1.5
1.4
MM0
2
1
cvtpi2ps xmm0, mm0
127
95
63
31
0
MM0
2
1
XMM0
a3
a2
2.0
1.0

Векторные расширения



- 1997 - MMX (Intel Pentium MMX)
  - 8 байт, целочисленные операции
  - 8 регистров, mm0-mm7, поверх FPU
  - арифметика с насыщением
- 1998 - 3DNow! (AMD K6-2)
  - 8 байт, float
  - поверх MMX, FPU

Векторные расширения



- 1999 - SSE (Pentium III)
  - 16 байт, float
  - 8(16) регистров xmm0-xmm7 (xmm15)
- 2001 - SSE2 (Pentium 4)
  - 16 байт, double
- 2004 - SSE3 (Pentium 4 Prescott)
  - горизонтальные инструкции
- 2006 - SSSE3 (Core 2, Atom)
  - расширение целочисленного набора
- 2007 - SSE4.1 (Penryn)
- 2008 - SSE4.2 (Nehalem)
  - видео, строки, криптография

Векторные расширения



- 2011 - AVX (Sandy Bridge)
  - 32 байта, float, double
  - трехоперандные команды
- 2013 - AVX2 (Haswell)
  - 32 байта, целочисленные
  - FMA: r = a * b + c
- 2016 - AVX-512 (KNL)
  - 64 байта

AVX / AVX2



- Регистры: ymm
- маскированная загрузка и сохранение
- широковещательная загрузка
- AVX:
  - для ряда инструкций выравнивание обязательно (S)
- AVX2:
  - добавлены целочисленные инструкции
- инструкции VEXTRACTF128, VINSERTF128

FMA (Fused Multiply Add)



- C = C **A + B, C** = **A** B + C
- A*B+C, A*B-C, -(A*B)+C, -(A*B)-C

Составные части AVX

Операция	**Инструкции**
Вертикальные арифметические	`addpd, mullpd, divpd`
Горизонтальные арифметические	`haddpd, hsubpd`
Разделение	
Слияния (Blend)	`BLENDPD, BLENDVPD`
Broadcast (Avx)	`VBROADCASTSD`
Сравнение	`CMPPD`
Маскированная загрузка и сохранение	`VMASKMOVPD`
Перестановка	`VPERMILPD`

AVX-512



- Группа F, CD, ER, PF
  - Xeon Phi x200 (Knights Landing)
  - Xeon Gold/Platinum (Skylake SP «Purley», не полная поддержка)
- AVX-512 Foundation (F)
  - Расширения 32- и 64-битных инструкций AVX (маски, broadcasting, округления и генерации исключений)
- AVX-512 Conflict Detection Instructions (CD)
- AVX-512 Exponential and Reciprocal Instructions (ER)
  - exponential and reciprocal operations designed to help implement transcendental operations
- AVX-512 Prefetch Instructions (PF)

AVX-512



- Группа VL, DQ, BW
  - Skylake X
  - Cannon Lake
- AVX-512 Vector Length Extensions (VL)
  - Расширение операций AVX-512 на взаимодействие с регистрами XMM (128-bit) и YMM (256-bit)
- AVX-512 Doubleword and Quadword Instructions (DQ)
  - Новые 32-bit и 64-bit инструкции
- AVX-512 Byte and Word Instructions (BW)
  - 8-bit и 16-bit целочисленные операции

AVX-512



- Группа VNNI, VBMI2, BITALG
  - Ice Lake
- AVX-512 Vector Neural Network Instructions (VNNI)
- AVX-512 Vector Byte Manipulation Instructions 2 (VBMI2)
  - byte/word load, store and concatenation with shift
- AVX-512 Bit Algorithms (BITALG)
  - byte/word bit manipulation instructions expanding VPOPCNTDQ

AVX-512



Name	Extension sets	Registers	Types
Legacy SSE	SSE-SSE4.2	`xmm0-xmm15`	single floats. From SSE2: bytes, words, doublewords, quadwords and double floats.
AVX-128 (VEX)	AVX, AVX2	`xmm0-xmm15`	bytes, words, doublewords, quadwords, single floats and double floats.
AVX-256 (VEX)	AVX, AVX2	`ymm0-ymm15`	single float and double float. From AVX2: bytes, words, doublewords, quadwords.
AVX-128 (EVEX)	AVX-512VL	`xmm0-xmm31 (k0-k7)`	doublewords, quadwords, single float and double float. With AVX512BW:

			bytes and words. With AVX512-FP16: half float.
AVX-256 (EVEX)	AVX-512VL	`ymm0-ymm31 (k0-k7)`	doublewords, quadwords, single float and double float. With AVX512BW: bytes and words. With AVX512-FP16: half float.
AVX-512 (EVEX)	AVX-512F	`zmm0-zmm31 (k0-k7)`	doublewords, quadwords, single float and double float. With AVX512BW: bytes and words. With AVX512-FP16: half float.

Организация регистров

512	256	128
ZMM0	YMM0	XMM0
ZMM1	YMM1	XMM1
...	...	...
ZMM31	YMM31	XMM31

ПРАКТИЧЕСКИЕ ПРИМЕРЫ

Примеры кода (1)



```
unsigned __int64 start = __rdtsc();
for (unsigned i = 0; i < SIZE; i++) {
    arr_c[i] = arr_a[i] + arr_b[i];
}
unsigned __int64 time = __rdtsc() - start;
printf("Execution time (1): %I64d ticks\n", time);
```

Вывод о компиляции



- Флаг: /Qvec-report2
- 1> — Analyzing function: main
- 1> main.cpp(18) : info C5001: loop vectorized
- 1> main.cpp(25) : info C5001: loop vectorized
- 1> main.cpp(33) : info C5001: loop vectorized
- 1> main.cpp(40) : info C5002: loop not vectorized due to reason „1200“

Разбор в asm-листинг



```
; 20  :    arr_c[i] = arr_a[i] + arr_b[i];
movss xmm0, DWORD PTR [rcx+rax-4]
mulss xmm0, DWORD PTR [rax-4]
lea rax, QWORD PTR [rax+16]
movss DWORD PTR [rdx+rax-20], xmm0
movss xmm1, DWORD PTR [rax+rcx-16]
mulss xmm1, DWORD PTR [rax-16]
movss DWORD PTR [rax+rdx-16], xmm1
movss xmm0, DWORD PTR [rcx+rax-12]
mulss xmm0, DWORD PTR [rax-12]
movss DWORD PTR [rdx+rax-12], xmm0
movss xmm1, DWORD PTR [rcx+rax-8]
mulss xmm1, DWORD PTR [rax-8]
movss DWORD PTR [rdx+rax-8], xmm1
```

Разбор в asm-листинг



```
$LL29@main:
; 19  :    for (unsigned i = 0; i < SIZE; i++) {
movups xmm0, XMMWORD PTR [rax-16]
movups xmm1, XMMWORD PTR [rcx+rax-16]
lea rax, QWORD PTR [rax+32]
; 20  :    arr_c[i] = arr_a[i] + arr_b[i];
mulps xmm1, xmm0
movups XMMWORD PTR [rdx+rax-48], xmm1
movups xmm2, XMMWORD PTR [rcx+rax-32]
movups xmm0, XMMWORD PTR [rax-32]
mulps xmm2, xmm0
movups XMMWORD PTR [rdx+rax-32], xmm2
sub rdi, 1
jne SHORT $LL29@main
```

Примеры кода (2)



```
unsigned __int64 start = __rdtsc();
for (unsigned i = 0; i < SIZE; i++) {
    arr_c[i] = arr_a[i] \* 3.f;
}
unsigned __int64 time = __rdtsc() - start;
printf("Execution time (2): %I64d ticks\n", time);
```

Разбор в asm-листинг



```
$LN66@main:
movaps xmm2, XMMWORD PTR __xmm@40400000404000004040000040400000
; 29  :    arr_c[i] = arr_a[i] * 3.f;
mulps xmm0, xmm2
movups XMMWORD PTR [rax-48], xmm0
movups xmm1, XMMWORD PTR [rdi+rax-32]
mulps xmm1, xmm2
movups XMMWORD PTR [rax-32], xmm1
```

Примеры кода (3)



```
unsigned __int64 start = __rdtsc();
for (unsigned i = 0; i < SIZE; i++) {
    arr_c[i] = arr_a[i] + arr_b[SIZE - i - 1];
}
unsigned __int64 time = __rdtsc() - start;
printf("Execution time (3): %I64d ticks\n", time);
```

Разбор в asm-листинг



```
; 38  :    arr_c[i] = arr_a[i] + arr_b[SIZE - i - 1];
mov eax, 1048575 ; 000fffffH
lea rcx, QWORD PTR [rcx+32]
sub eax, edx
movss xmm0, DWORD PTR [rsi+rax*4]
mov eax, 1048574 ; 000ffffeH
mulss xmm0, DWORD PTR [rdi+rcx-36]
sub eax, edx
movss DWORD PTR [rcx-36], xmm0
movss xmm0, DWORD PTR [rsi+rax*4]
mov eax, 1048573 ; 000ffffdH
mulss xmm0, DWORD PTR [rdi+rcx-32]
sub eax, edx
movss DWORD PTR [rcx-32], xmm0
movss xmm0, DWORD PTR [rsi+rax*4]
cmp edx, 1048576 ; 00100000H
jb $LL13@main
```



ВЕКТОРНЫЕ АРХИТЕКТУРЫ

Примеры векторных процессоров

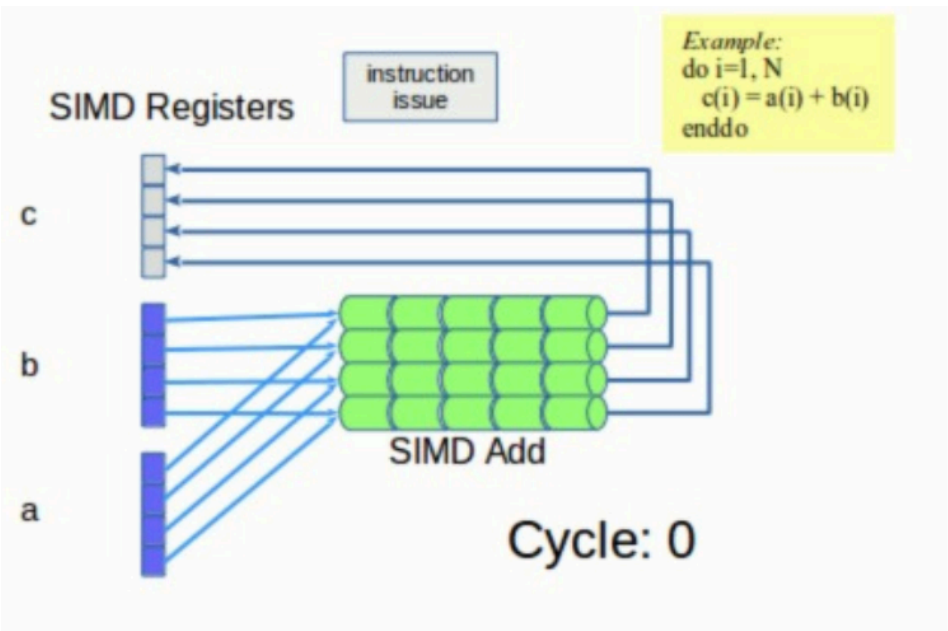
Processor (year)	Vector clock rate (MHz)	Vector registers	Elements per register (64-bit elements)	Vector arithmetic units	Vector load-store units	Lanes
Cray-1 (1976)	80	8	64	6: FP add, FP multiply, FP reciprocal, integer add, logical, shift	1	1
Cray X-MP (1983) Cray Y-MP (1988)	118 166	8	64	8: FP add, FP multiply, FP reciprocal, integer add, 2 logical, shift, population count/parity	2 loads 1 store	1
Cray-2 (1985)	244	8	64	5: FP add, FP multiply, FP reciprocal/sqrt, integer add/shift/population count, logical	1	1
Fujitsu VP100/ VP200 (1982)	133	8–256	32–1024	3: FP or integer add/logical, multiply, divide	2	1 (VP100) 2 (VP200)
Hitachi S810/S820 (1983)	71	32	256	4: FP multiply-add, FP multiply/divide-add unit, 2 integer add/logical	3 loads 1 store	1 (S810) 2 (S820)
Convex C-1 (1985)	10	8	128	2: FP or integer multiply/divide, add/logical	1	1 (64 bit) 2 (32 bit)
NEC SX/2 (1985)	167	8+32	256	4: FP multiply/divide, FP add, integer add/ logical, shift	1	4
Cray C90 (1991) Cray T90 (1995)	240 460	8	128	8: FP add, FP multiply, FP reciprocal, integer add, 2 logical, shift, population count/parity	2 loads 1 store	2
NEC SX/5 (1998)	312	8+64	512	4: FP or integer add/shift, multiply, divide, logical	1	16
Fujitsu VPP5000 (1999)	300	8–256	128–4096	3: FP or integer multiply, add/logical, divide	1 load 1 store	16
Cray SV1 (1998) SV1ex (2001)	300 500	8	64 (MSP)	8: FP add, FP multiply, FP reciprocal, integer add, 2 logical, shift, population count/parity	1 load-store 1 load	2 8 (MSP)
VMIPS (2001)	500	8	64	5: FP multiply, FP divide, FP add, integer add/ shift, logical	1 load-store	1
NEC SX/6 (2001)	500	8+64	256	4: FP or integer add/shift, multiply, divide, logical	1	8
NEC SX/8 (2004)	2000	8+64	256	4: FP or integer add/shift, multiply, divide, logical	1	4
Cray X1 (2002) Cray XIE (2005)	800 1130	32	64 256 (MSP)	3: FP or integer, add/logical, multiply/shift, divide/square root/logical	1 load 1 store	2 8 (MSP)

Векторные процессоры

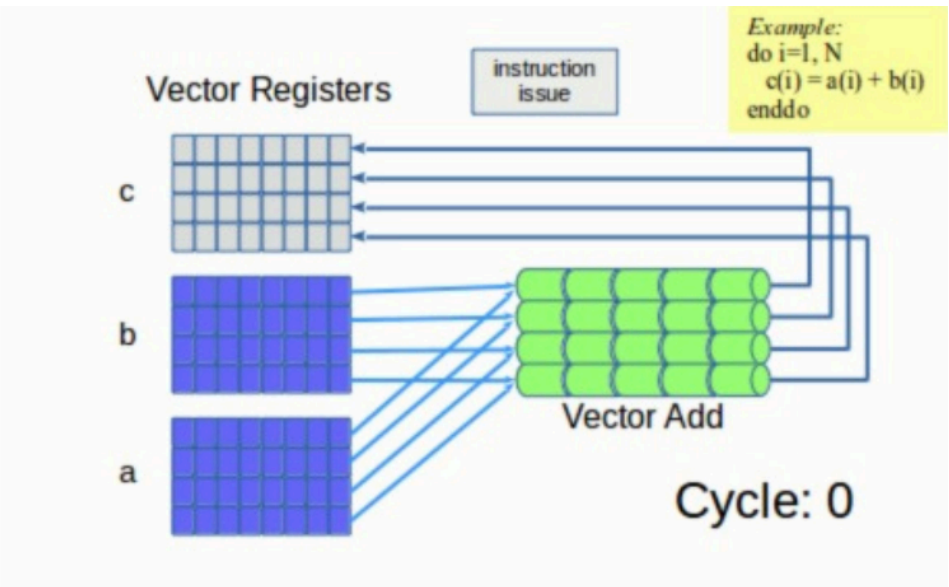
- Суперкомпьютер NEC SX-Aurora TSUBASA
  - 2017 год
  - 8 ядер
    - частота: 1.4 - 1.6 ГГц
    - пиковая производительность: 2.15 - 2.45 TFLOPS



Особенности NEC



SIMD



Vector

NEC Vector Engine Core

Vector Processing Unit (VPU)

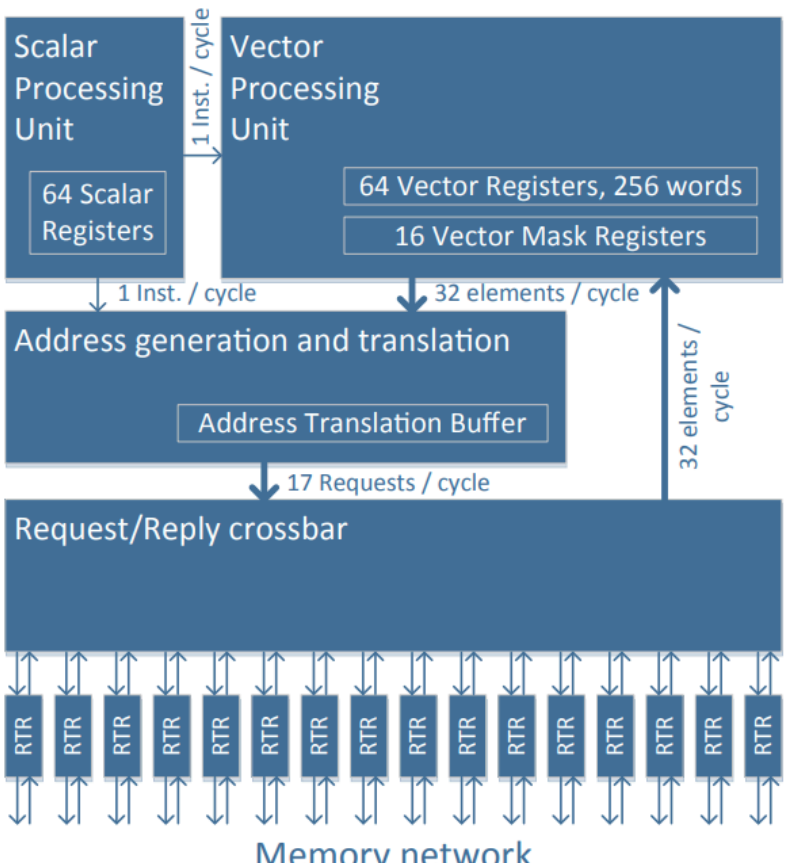
- Powerful
  - 307.2GFLOPS DP / 614.4GFLOPS SP performance
- High bandwidth memory access
  - 409.6GB/sec Load and Store

Scalar Processing Unit (SPU)

- Provides basic processor functions
  - Fetch, decode, branch, add, exception handling, etc…
- Controls the status of complete core

Address translation and data forwarding crossbar

- To support contiguous vector memory access
  - 16 elements/cycle vector address generation and translation,
    17 requests/cycle issuing
  - 409.6GB/sec load and 409.6GB/sec store data forwarding



NEC Vector Processor Unit

- Four pipelines, each 32-way parallel
  - –FMA0: FP fused multiply-add, integer multiply
  - –FMA1: FP fused multiply-add, integer multiply
  - –ALU0/FMA2: Integer add, multiply, mask, FP FMA
  - –ALU1/Store: Integer add, store, complex operation

Total 96 FMAs

- Doubled SP performance by 32bit x 2 packed vector data support
- Vector register (VR) renaming with 192 physical VRs
  - 64 architectural VRs are renamed
    - –Enhanced preload capability
    - –Avoidance of WAR and WAW dependencies
- OoO scheduling
- Dedicated complex operation pipeline to prevent pipeline stall
  - Vector sum, divide, mask population count, etc.

